

SALESTORM — Requirements & Assumptions

1. Business Context

SALESTORM is designed for a high-demand flash sale where a limited number of products become available to a large number of customers at the same time.

For this scenario, only **100 units** are available while up to **10,000 customers may attempt to purchase concurrently**. Normal traffic is assumed to be around **10,000 requests per second**, with the architecture designed to handle a flash-sale peak of up to **500,000 requests per second at the edge**.

The main challenge is not only handling the traffic. The system must maintain correct inventory, prevent duplicate purchases, safely handle payment failures and retries, and ensure that a successful payment is not lost when a downstream service becomes temporarily unavailable.

Scenario Parameters

Parameter	Value
Available stock	100 units
Concurrent buyers	10,000
Normal traffic	~10,000 requests/sec
Flash-sale peak	Up to 500,000 requests/sec at the edge
Payment success rate	95%
Duplicate requests	~2%
Order Service outage	30 seconds
Reservation validity	10 minutes
Purchase limit	1 unit per customer

2. Functional Requirements

ID	Requirement
FR1	Customers can view the flash-sale product, price, sale status, and remaining stock.
FR2	Customers can initiate a purchase through a purchase API using an Idempotency-Key .
FR3	The system must reserve inventory atomically so that the same unit cannot be reserved by multiple customers.
FR4	A successful reservation must remain valid for a maximum of 10 minutes while payment is completed.
FR5	Payments must be processed through an external payment gateway using tokenised payment information.
FR6	If payment definitively fails or the reservation expires, the reserved inventory must be released.
FR7	Payment timeouts or uncertain gateway responses must be treated as UNKNOWN until the final status is confirmed.
FR8	After confirmed payment success, an order must be created asynchronously.
FR9	A successful payment must not be lost if the Order Service is temporarily unavailable.
FR10	Customers must be able to retrieve their current order status.
FR11	The system must support asynchronous events for fulfilment, shipment, and customer notifications.
FR12	Administrators must be able to configure the flash sale and monitor system and simulation metrics.

3. Non-Functional Requirements

ID	Requirement	Target
NFR1	Inventory Consistency	Zero overselling; inventory must never become negative.
NFR2	Transaction Uniqueness	No duplicate reservation, payment, or order for the same purchase.
NFR3	Payment Durability	A confirmed payment must not result in a permanently lost order.
NFR4	Scalability	Stateless services must scale horizontally and the edge must support the defined flash-sale traffic.
NFR5	Reservation Latency	p99 target below 200 ms under the supported load.
NFR6	Read Latency	Cached product and stock reads should target p99 below 50 ms.
NFR7	Availability	99.9% availability for the checkout path with multi-AZ deployment.
NFR8	Fault Tolerance	Temporary service, database, broker, and payment failures must not violate inventory or payment consistency.
NFR9	Security	TLS, authentication, RBAC, WAF protection, input validation, and no storage of raw card data.
NFR10	Observability	Metrics, structured logs, distributed tracing, and correlation IDs across the purchase flow.

4. Assumptions

4.1 Inventory

- SQL is the **authoritative source of truth** for inventory.
- Each inventory shard has a single authoritative primary with a synchronous standby in a second availability zone.
- Inventory reservations are performed using an atomic database operation.
- Redis may be used for caching and traffic management, but it is **never the source of truth for inventory**.

4.2 Payment

- The external payment gateway supports idempotency keys and payment-status lookup.
- The payment gateway may receive the same request more than once, so payment processing must be idempotent.
- A gateway timeout does not necessarily mean that payment failed.
- Uncertain payment results are reconciled using the gateway's status API.

4.3 Messaging

- The message broker provides durable, at-least-once delivery.
- The same event may be delivered more than once.
- Event consumers must therefore be idempotent.

4.4 Traffic

- The **500,000 requests/second** figure represents traffic handled at the edge; it does not mean that 500,000 inventory writes occur every second.
- With only 100 available units, successful reservations are naturally limited by available inventory.
- Requests that cannot be fulfilled should be rejected or controlled before unnecessarily reaching the inventory database.

4.5 Users

- Customers are authenticated before making a purchase.
- Each customer can purchase a maximum of **one unit per flash-sale event**.

5. Constraints & Business Rules

5.1 Inventory Rules

- Inventory must never become negative.
- A reservation can succeed only when stock is available.
- Inventory reservation must be atomic.
- Each reservation is valid for **10 minutes**.
- An expired reservation must release its inventory.
- Reservation release must be safe to retry without releasing the same inventory more than once.

5.2 Idempotency Rules

- Every purchase request must contain an **Idempotency-Key**.
- The same key represents the same logical purchase operation.
- Retrying the same request must return the existing result instead of creating another reservation.

- Reusing an idempotency key with a different request payload must be rejected.
- One reservation can create only one payment.
- One successful payment can create only one order.

5.3 Payment Rules

- Payment requests must be idempotent.
- SUCCEEDED → continue to order creation.
- FAILED → release the reservation.
- UNKNOWN → do not immediately release the reservation; reconcile with the payment gateway.
- A retry must never result in the customer being charged twice for the same purchase.

5.4 Order Rules

- An order is created only after payment is confirmed.
- Payment success must be durably recorded before the order-creation event is published.
- If the Order Service is unavailable, the event must remain available for processing after recovery.
- Order creation must be idempotent so that event retries cannot create duplicate orders.

6. Failure Scenarios

The architecture must safely handle the following scenarios:

Scenario	Expected Behaviour
Successful purchase	Inventory is reserved → payment succeeds → order is created.
Payment failure	Payment is marked failed → reservation is released → inventory becomes available again.
Payment timeout	Payment is marked UNKNOWN → status is reconciled with the gateway.
Duplicate purchase request	Idempotency prevents duplicate reservations and payments.
Inventory reaches zero	Further purchase attempts are rejected without creating reservations.
Order Service unavailable for 30 seconds	Successful payment events remain durable and are processed after the service recovers.
Database failure	Configured database failover maintains inventory consistency.

Duplicate broker event	Idempotent consumers prevent the same business action from being performed twice.
Reservation expiry	Reservation is released and the inventory becomes available again.
Sudden traffic spike	Edge controls, caching, rate limiting, and horizontal scaling protect the critical purchase path.

7. Core Business Invariants

Regardless of traffic, retries, or temporary failures, the following conditions must always remain true:

1. The system must never sell more units than are available.
2. Inventory must never become negative.
3. One purchase request must not create multiple reservations.
4. One reservation must not result in multiple successful payments.
5. One successful payment must not create multiple orders.
6. A successful payment must not be permanently lost because the Order Service is unavailable.
7. A payment timeout must not be treated as an immediate payment failure.
8. Retries must be safe and must not create duplicate business operations.
9. Failure of Redis or another cache must not corrupt the authoritative inventory state.

8. Out of Scope

To keep the solution focused on the flash-sale problem, the following are outside the scope of this design:

- Product recommendation systems
- Large-scale product catalogue management
- Personalised recommendations
- Full logistics or delivery-provider implementation
- Advanced storefront UI/UX
- Marketing and campaign management
- Loyalty and reward systems
- Warehouse management
- Multi-product cart optimisation
- International tax and payment processing
- Storage of raw card or payment credentials

9. Success Criteria

The architecture will be considered successful if it can demonstrate that:

- **100 available units never result in more than 100 successful reservations.**
- Duplicate purchase requests do not create duplicate transactions.
- Failed payments correctly release reserved inventory.
- Payment timeouts are safely reconciled rather than incorrectly treated as failures.
- Successful payments are not lost during the **30-second Order Service outage**.
- Inventory remains consistent during high-concurrency purchase attempts.
- Critical events can be retried safely without creating duplicate orders or payments.
- The architecture can absorb the defined flash-sale traffic at the edge without overwhelming the inventory database.
- Security, reliability, and observability are considered as part of the architecture from the beginning.